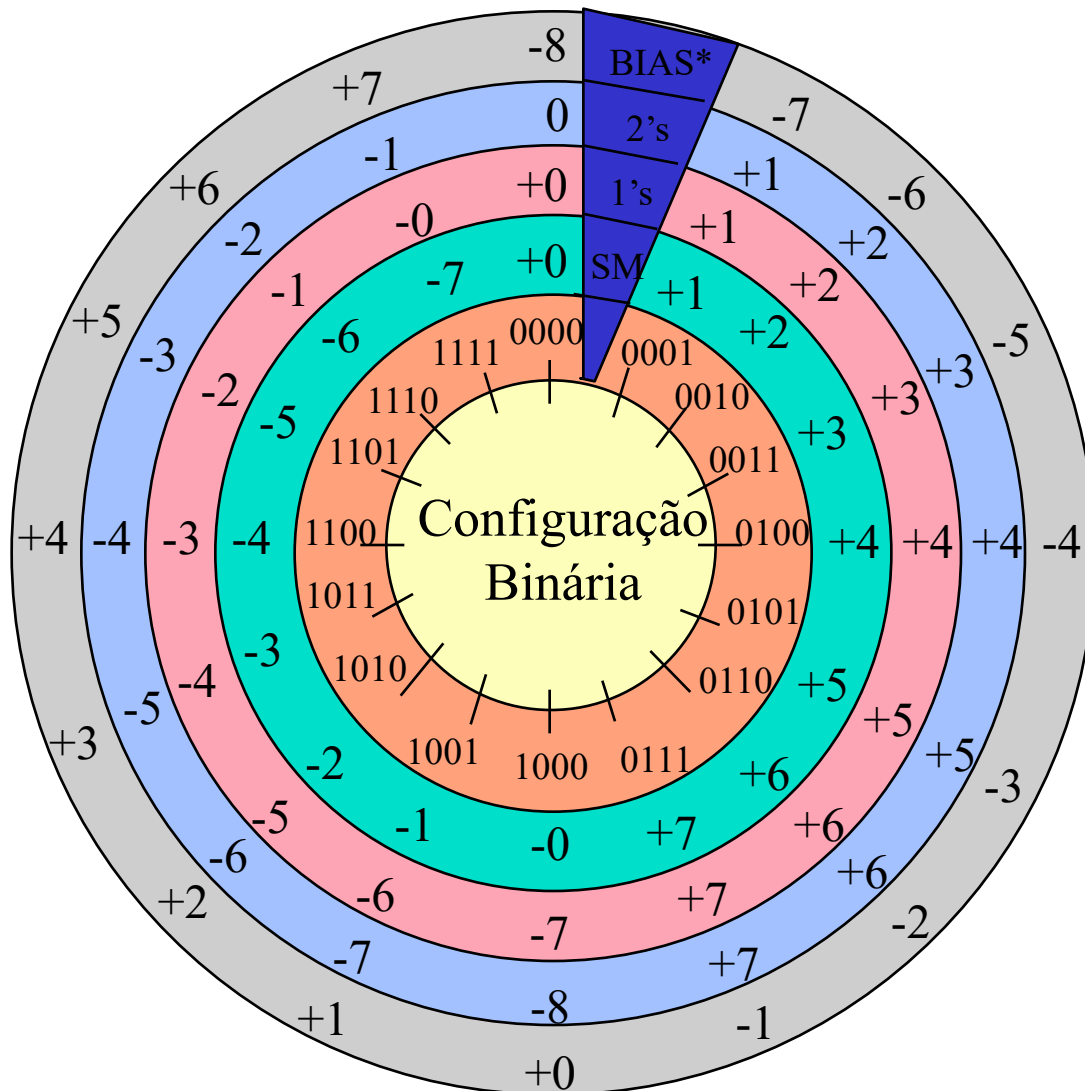


Multiplicação e Divisão Serial

Conceitos básicos

Moraes/Ney - 13/novembro/2018

Representações de Inteiros - 4 formas



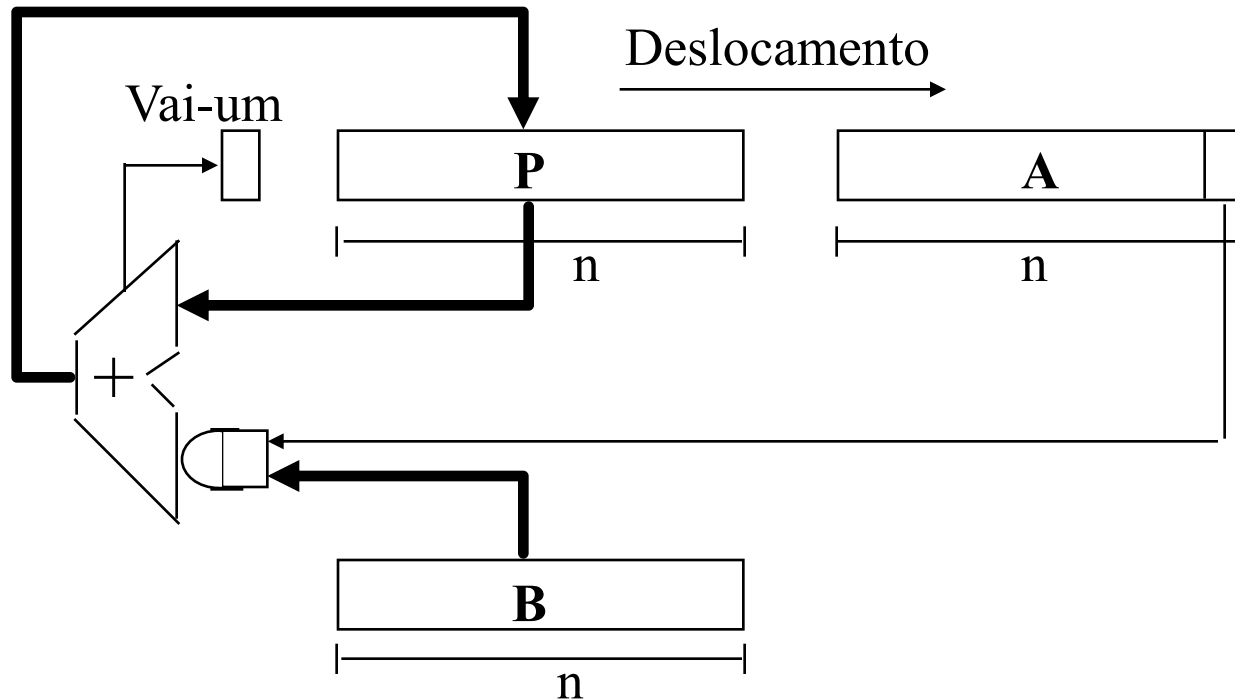
-Exemplo para 4 bits

- * polarização = $2^{(n-1)} = 8$

- ➔ Bias tem distribuição uniforme com relação a binários puros;
- ➔ Complemento de 2 facilita soma;
- ➔ Complemento de 1 facilita complementação;
- ➔ SM é fácil de entender e separa sinal de valor;

Multiplicação serial

➡ Solução para $a \cdot b$: somas sucessivas n passos



- ➡ Inicialmente, $P=0$, $A=a$, $B=b$. Cada passo, duas partes:
- soma carregada em **P**;
 - **P & A** deslocado um bit para a direita.

Multiplicação A*B

A = 11011 (27)

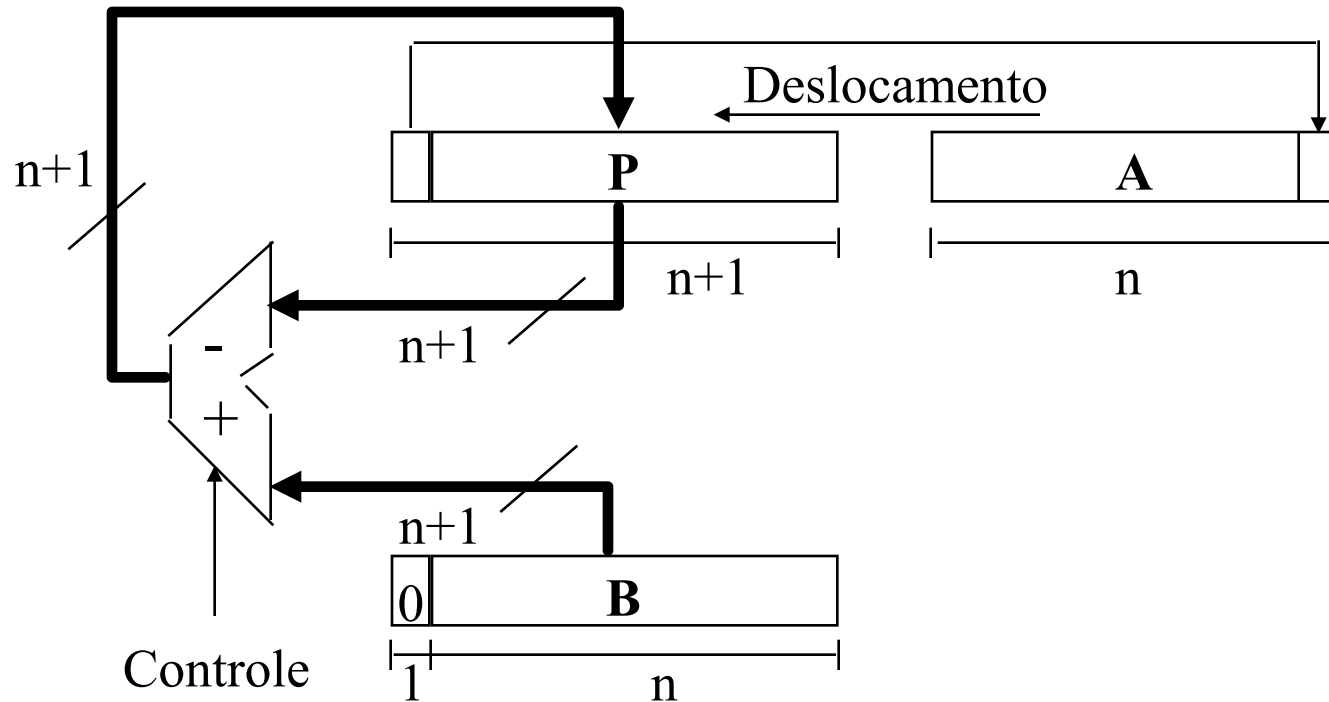
B = 00101 (5)

135 → 100 00111

		P					A				
passo	0	0	0	0	0	0	1	1	0	1	1
1	0	0	0	1	0	1	1	1	0	1	1
	0	0	0	0	1	0	1	1	1	0	1
2	0	0	0	1	1	1	1	1	1	0	1
	0	0	0	0	1	1	1	1	1	1	0
3	0	0	0	0	1	1	1	1	1	1	0
	0	0	0	0	0	1	1	1	1	1	1
4	0	0	0	1	1	0	1	1	1	1	1
	0	0	0	0	1	1	0	1	1	1	1
5	0	0	1	0	0	0	0	1	1	1	1
	0	0	0	1	0	0	0	0	1	1	1

Divisão serial

- Solução para a/b : subtrações sucessivas, n passos



- Cada passo, quatro partes: **1)** desloca P&A p/ esq 1 bit; **2)** $P \leftarrow P - B$; **3)** If (passo 2 < 0), $A_0 = 0$ else $A_0 = 1$; **4)** If (passo 2 < 0), restaura P fazendo $P \leftarrow P + B$.

Divisão A/B

A = 11011 (27)

B = 00101 (5)

- 1) desloca P&A p/ esq 1 bit;
- 2) $P \leftarrow P-B$;
- 3) If ($P < 0$), $A_0 = 0$ else $A_0 = 1$;
- 4) If ($P < 0$), restaura P fazendo $P \leftarrow P+B$

		P (conterá o resto)					A (conterá a divisão)				
passo	0	0	0	0	0	0	1	1	0	1	1
1	0	0	0	0	0	1	1	0	1	1	0
	0	0	0	0	0	1	1	0	1	1	0
2											
3											
4											
5											

Divisão A/B

A = 11011 (27)

B = 00101 (5)

- 1) desloca P&A p/ esq 1 bit;
- 2) $P \leftarrow P - B$;
- 3) If ($P < 0$), $A_0 = 0$ else $A_0 = 1$;
- 4) If ($P < 0$), restaura P fazendo $P \leftarrow P + B$

		P (conterá o resto)					A (conterá a divisão)				
passo	0	0	0	0	0	0	1	1	0	1	1
1	0	0	0	0	0	1	1	0	1	1	0
	0	0	0	0	0	1	1	0	1	1	0
2	0	0	0	0	1	1	0	1	1	0	0
	0	0	0	0	1	1	0	1	1	0	0
3											
4											
5											

Divisão A/B

A = 11011 (27)

B = 00101 (5)

- 1) desloca P&A p/ esq 1 bit;
- 2) $P \leftarrow P - B$;
- 3) If ($P < 0$), $A_0 = 0$ else $A_0 = 1$;
- 4) If ($P < 0$), restaura P fazendo $P \leftarrow P + B$

		P (conterá o resto)					A (conterá a divisão)				
passo	0	0	0	0	0	0	1	1	0	1	1
1	0	0	0	0	0	1	1	0	1	1	0
	0	0	0	0	0	1	1	0	1	1	0
2	0	0	0	0	1	1	0	1	1	0	0
	0	0	0	0	1	1	0	1	1	0	0
3	0	0	0	1	1	0	1	1	0	0	0
	0	0	0	0	0	1	1	1	0	0	1
4											
5											

00110 - 00101 = 001

Divisão A/B

A = 11011 (27)

B = 00101 (5)

1) desloca P&A p/ esq 1 bit;

2) $P \leftarrow P - B$;

3) If ($P < 0$), $A_0 = 0$ else $A_0 = 1$;

4) If ($P < 0$), restaura P fazendo $P \leftarrow P + B$

		P (conterá o resto)					A (conterá a divisão)				
passo		0	0	0	0	0	1	1	0	1	1
1	0	0	0	0	0	1	1	0	1	1	0
	0	0	0	0	0	1	1	0	1	1	0
2	0	0	0	0	1	1	0	1	1	0	0
	0	0	0	0	1	1	0	1	1	0	0
3	0	0	0	1	1	0	1	1	0	0	0
	0	0	0	0	0	1	1	1	0	0	1
4	0	0	0	0	1	1	1	0	0	1	0
	0	0	0	0	1	1	1	0	0	1	0
5	0	0	0	1	1	1	0	0	1	0	0
	0	0	0	0	1	0	0	0	1	0	1

Resto = 2

resultado=5

VHDL

```

entity divide is
    generic(N: integer := 16);
    port(divisor:          in  std_logic_vector((N-1) downto 0);
          dividendo:      in  std_logic_vector((N-1) downto 0);
          clock,start:    in  std_logic;
          endop:          out std_logic;
          divisao:        out std_logic_vector(N-1 downto 0);
          resto:          out std_logic_vector(N-1 downto 0));
end;

architecture divide of divide is
    type State_type is (inicializa, desloca, calc, termina, fim);
    signal EA: State_type;

    signal regP:          std_logic_vector(N*2 downto 0);
    signal regB:          std_logic_vector(N   downto 0);
    signal diferenca:     std_logic_vector(N   downto 0);
    signal cont:          integer;

begin

```

INICIALIZAÇÕES

```

regP(N*2 downto N) <= (others=>'0');
regP(N-1 downto 0) <= divisor;
regB(N) <= '0';
regB(N-1 downto 0) <= dividendo;

```

atribuição

```

diferenca <= regP(N*2 downto N) - regB(N downto 0);
regP(0) <= '1';
regP(N*2 downto N) <= diferenca;

```

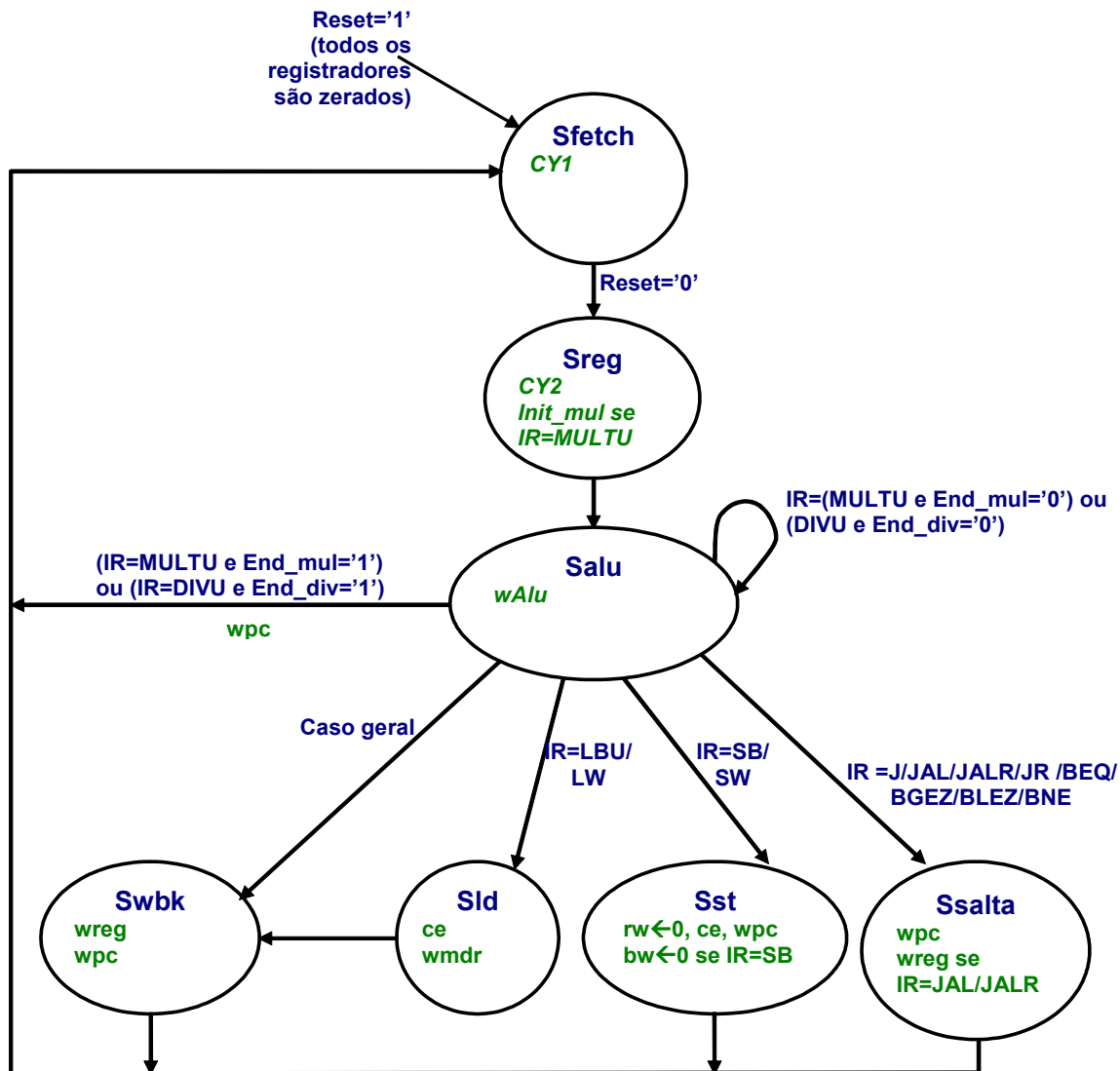
rotação

```

regP <= regP(N*2-1 downto 0) & regP(N*2);

```

Integração ao MIPS – bloco de controle



Integração ao MIPS – bloco de dados

